

DaphOS Backend Coding Challenge – Employee Time Tracking API

DaphOS ist eine Softwareplattform, die Krankenhäuser und Gesundheitseinrichtungen dabei unterstützt, ihre Prozesse effizienter zu planen und zu steuern. Unser Ziel ist es, komplexe Abläufe – von der Personalplanung bis zum Klinikmanagement – durch smarte, nutzerfreundliche Anwendungen zu vereinfachen.

Im Rahmen unseres Bewerbungsprozesses möchten wir dich mit einer kleinen Coding Challenge einladen, uns zu zeigen, wie du an technische Problemstellungen herangehst – von der Struktur über Logik bis zur Code-Qualität.

Keine Sorge: Es geht nicht um Perfektion, sondern um deinen **Ansatz, deine Denkweise und deinen Stil**.

Ziel der Aufgabe

Erstelle eine kleine REST-API, mit der Arbeitszeiten von Mitarbeiter:innen erfasst und ausgewertet werden können.

Die Anwendung soll ermöglichen:

- Mitarbeiter:innen zu verwalten (CRUD)
- Arbeitszeiten für Mitarbeiter:innen zu erfassen (CRUD)
- Sinnvolle Validierungen einzubauen (z. B. keine überlappenden Schichten)
- Auswertungen pro Mitarbeiter:in bereitzustellen (Summary Endpoint)
- Bonus: Eine Statistik über alle Mitarbeiter:innen anzubieten

Technischer Rahmen

Du kannst die Aufgabe in der Sprache und mit dem Framework deiner Wahl umsetzen – zum Beispiel:

- Python (FastAPI, Django)
- Java (Spring Boot)
- Go, Rust oder Node.js

Als Datenbank kannst du SQLite oder PostgreSQL verwenden.

Die Daten sollen dauerhaft gespeichert werden (kein In-Memory-Ansatz).

Domänenmodell

Überlege dir ein minimales Domänenmodell, mit dem du die Use Cases umsetzen kannst.

API-Endpunkte

Überlege dir eine REST-Schnittstelle als API mit denen du die Use Cases umsetzen kannst.

Abgabe / Deliverables

Bitte reiche dein Ergebnis als GitHub-Repository oder ZIP-Ordner ein.

Dein Projekt sollte enthalten:

- Quellcode
- README mit Setup-Anleitung und Beispiel-Requests
- Datenbankschema oder Migrationsskript
- Optional: Dockerfile und Tests

Deine Anwendung soll lokal lauffähig sein und sich mit einer kurzen Anleitung starten lassen (z. B. in einer README beschrieben oder via Docker).

Nicht nötig / Non-Goals

Folgende Punkte sind nicht Teil der Aufgabe und müssen daher nicht umgesetzt werden:

- Benutzeroberfläche (UI) oder Frontend-Komponenten
- Authentifizierung

Bewertungskriterien

Wir achten besonders auf:

- Verständliche Struktur & sauberer Code
- Sinnvolle API-Design-Entscheidungen
- Lesbarkeit und Nachvollziehbarkeit
- Optional: Testabdeckung, Dokumentation, saubere Architektur

Ablauf

1. Take-Home Challenge (max. 4-5 Stunden, timeboxed) – du arbeitest selbstständig an der Aufgabe und schickst uns das Ergebnis.
2. Follow-up Gespräch (60 Min.) – wir gehen gemeinsam durch deinen Code, sprechen über Design-Entscheidungen und bauen ggf. gemeinsam eine kleine Erweiterung ein.